A New Path Planning Algorithm For Moving A Point Object Amidst Unknown Obstacles In A Plane [1]

A. Sankaranarayanan and M. Vidyasagar

Department of Electrical Engineering
University of Waterloo, Waterloo
Ontario N2L 3G1, Canada

ABSTRACT

A nonheuristic path planning of moving a point object (*MA*) in a two dimensional plane, amidst unknown obstacles, is considered. A path is to be generated, point by point, using only the local information like, *MA's* current position and whether it is in contact with an obstacle. The two existing algorithms which solve this problem are due to Lumelsky and Stepanov [9]. A new path planning algorithm to solve this problem is proposed. The motivation for the new algorithm is to realize the smallest worst case path length possible in its category (Sankar [13]). The procedure for the new algorithm is presented with explanations. Its various characteristics like, local cycle creation, worst case path length, target reachability conditions, etc., are dealt with. Its performance is compared with those of the existing algorithms. Examples on the operation of the algorithm are presented.

1. INTRODUCTION

A practical robotic application involves moving a robot (mobile or fixed to the floor) amidst obstacles, without colliding with them. This problem of obstacle avoidance has been extensively studied and has generated a vast literature. The research on path planning in the presence of obstacles can be classified into two large categories. In the first category [1-3], popularly known as Piano Movers' problem, an object (generally polyhedral) is to be moved in a scene filled with other polyhedral obstacles. A complete knowledge of the obstacles (i.e., their sizes, positions, orientations, etc.) is assumed. The scene, or a suitable transformation of it, is then divided into allowed and forbidden regions for the object's movement. A path is generated through a graph search, in a graph formed with these regions. This one-time, off-line and computationally very intensive procedure is most suitable for repetitive operations in an unchanging environment.

The second category of path planning deals with robot applications involving unknown (or partly known) and changing environments. The sensors (e.g., tactile) fitted on the robot, supply it with information on the obstacles in the immediate neighborhood of the robot. This information, naturally, can only be a local or a partial description of robot's environment. The robot's path is constructed on-line, point by point, using at any instant, only the local sensory information of the environment. A path based only on local information may not always exist, because the global requirement of moving a robot between two given points generally requires a global path planning with complete description of the environment (i.e., the first category). This results in one more classification. A path planning scheme of second category which can not always guarantee a path can be called, heuristical. Most of the schemes of second category, available in the literature, are heuristical. Some of them are discussed next.

Khatib [4] models a repulsive potential field around an obstacle to push the manipulator links away from the obstacles. Maciejewski and Klein [5], and Kircanski and Vukobratovic [6] propose similar schemes for a redundant manipulator. These schemes can not always guarantee a path, because the manipulator can either get trapped or keep oscillating between the obstacles, under certain conditions. Chattergy [7], proposes some heuristics to move a mobile robot fitted with sonar for proximity sensing, in an unknown environment. Numerous heuristical schemes for mobile robot navigation can be found in the literature.

A path planning scheme of second category which can always guarantee a path, when there exists one, can be called nonheuristical or provable. Some of the existing nonheuristical schemes are considered next. The Pledge Algorithm [8] generates a path for a point object to escape from a maze, whose global description is not known. It uses only the local information like, whether the point object is in contact with a wall. The problem studied in this paper was formulated by Lumelsky and Stepanov [9] and can be stated as follows:

Problem P1: *A mobile automaton (MA), which is a point object, has to be continuously moved between two given points S and T, in a plane filled with unknown obstacles of arbitrary shapes. The obstacle boundaries are simple closed curves. At any point on its path, MA knows only its current coordinates and those of target T. MA can store a few points from its path and use them for planning the future path sections. Also, MA can detect and follow an obstacle boundary when it meets one. No other information on the environment is available.*

A summary of the two nonheuristical algorithms, proposed by Lumelsky and Stepanov [9,10] to solve this problem, will be presented later in this paper. These algorithms are based on the Jordon Curve Theorem, and hence cannot generally be applied to environments of dimension greater than two. The application of these algorithms to 2-link manipulator obstacle

1 This research was supported by the Natural Sciences and Engineering Research Council of Canada under Grant A-1240.

CH2876-1/90/0000/1930$01.00 © 1990 IEEE

avoidance, taking into account the kinematics of different types of manipulators, is presented in Lumelsky [11,12].

This paper presents a new nonheuristical algorithm to solve the basic problem stated above. The motivation for the new algorithm, its procedure and its characteristics like, convergence, worst case path length, etc., are presented. The performance of the new algorithm is compared with those of the existing ones.

2. MODEL DEFINITION

The following definitions (mostly due to Lumelsky and Stepanov [9]), arising out of problem $P1$, model the scene and MA.

A scene (or environment) consists of obstacles and the points Start (S) and Target (T), all lying in a plane. The obstacle boundaries are simple closed curves of finite lengths. An obstacle boundary intersects any straight line only at finite number of points. Though, there is no restriction on the number of obstacles in a scene, any circular disc of finite radius is assumed to intersect only a finite number of obstacles. There is no restriction on the size and shape of an obstacle.

The *Mobile Automaton* (MA) is a point object. At any instant, MA, with respect to a reference point, knows its current coordinates and those of target T. The sensors on MA let it detect and follow an obstacle boundary, when one is met. MA, after hitting an obstacle can follow the obstacle boundary in two ways. Due to the absence any a priori information on the scene, it is not be possible to decide, whether MA should turn right or left to follow the obstacle boundary. Hence, for clarity and without loss of generality, the *local direction* at the obstacle boundary is assumed to be "left". The point at which MA meets an obstacle is called a hit point H. After moving along the obstacle boundary, MA may leave the obstacle boundary at a leave point L.

The path generated under an algorithm, say $Bug1$, is referred to as P_{Bug1} and its length is denoted as L_{Bug1}. p_i is the perimeter of the i_{th} obstacle met by MA on its way from S to T, and D is the length of the line segment ST. $d(x, y)$ is the Euclidean distance between two points x and y, belonging to the scene. $d_p(x, y)$ denotes the distance traveled by MA along its path (i.e., path length) while moving from a point x to a point y. Hence, $d_p(x, y)$ is not necessarily equal to $d_p(y, x)$. The path length along the path is determined by integrating the incremental lengths at every step. The total path length $d_p(S, x)$ up to a current position x is stored in the path length counter PC.

A path is said to contain a local cycle if MA visits any point on its path more than once.

3. EXISTING ALGORITHMS

A summary of the existing algorithms is presented for purposes of comparison with the new algorithm. For details, refer to Lumelsky and Stepanov [9,10].

3.1. Algorithm Bug1

$Bug1$ is a thorough algorithm in the sense, it leaves an obstacle only after traveling along its entire boundary. But, this also makes it conservative and overcautious.

Bug1 Procedure: MA moves along the line segment joining its present coordinates to T until an obstacle is met at a point H. At H, MA turns left and starts following the obstacle boundary until it returns to H. During this trip, the path length along the boundary from H is computed. The point Q, which is the closest on the boundary to T, and the path length from Q to H are also computed. After returning to H, MA goes back to Q along the shortest of the two possible boundary sections between H and Q. Then, it starts moving along the line segment QT. If an obstacle is met again, the above procedure is repeated. A target T is unreachable if the line segment QT crosses the obstacle at Q.

The upper bound on the path length (or worst case path length) generated by $Bug1$ is,

$$M_{Bug1} = 1.5\ \Sigma p_i + D, \tag{1}$$

where p_i and D are as defined before. An example of $Bug1$'s operation is shown in Fig. 3.

3.2. Algorithm Bug2

Under $Bug2$, MA generally leaves an obstacle boundary before it covers the entire boundary. This feature leads to shorter paths in some cases. This can also lead to local cycles and hence longer paths in other cases.

Bug2 Procedure: MA moves along the line segment ST until an obstacle is met at a point H. At H, MA turns left and follows the obstacle boundary until the line segment ST is met again at a point L. If L is the closest point to T on ST,, ever visited by MA, then MA starts moving along line segment LT. Otherwise, MA continues to move along the obstacle boundary until ST is met again. The *closest point* condition is again tested to decide whether MA should continue along the obstacle boundary or move along the line segment ST. This step eventually lets MA leave the obstacle boundary and proceed along ST. While moving along ST, if an obstacle is met (this could be one of the obstacles visited before), the procedure described is repeated. If MA returns to a hit point H, without defining a new leave point L, then target T is unreachable.

The upper bound on the path length generated by $Bug2$ is,

$$M_{Bug2} = \Sigma n_i p_i/2 + D, \tag{2}$$

where n_i is the number of intersections the line segment ST makes with i_{th} obstacle of perimeter p_i. An example of $Bug2$'s operation is shown in Fig. 4.

4. THE NEW ALGORITHM ALG1

4.1. Motivation

Algorithm $Bug1$, though it ensures that MA does not visit a point on an obstacle boundary more than twice, generally produces longer paths as MA has to go around the entire boundary of each obstacle it meets. For example, in a scene consisting of only convex obstacles, the path produced by $Bug1$ is very long.

For this scene and for other simple scenes, *Bug2* produces significantly shorter paths. This is so, because *MA* goes around only a part of each obstacle boundary and there are no local cycles created. But, for a scene which creats local cycles, *Bug2* can produce very long paths, while *Bug1* produces relatively shorter paths. The examples shown in Figs. 3 and 4, illustrate this point. Under *Bug2*, *MA* can visit a point on an obstacle boundary up to $n/2$ times, where n is the number of intersections the line segment ST makes with that obstacle. These repeated trips along sections of the path cause the longer worst case path length given in Eqn. 2.

It is desirable to form an algorithm which performs like *Bug2* for simple scenes, but at the same time, does not produce very long paths when complex scenes cause local cycles. The following result suggests that such an algorithm may actually exist: The worst case path length of an algorithm, in which *MA* leaves an obstacle without traveling along its entire boundary, has to be greater than or equal to $2\ \Sigma p_i + D$ (Sankar [13]). That means, an algorithm with only a worst case path length of $2\ \Sigma p_i + D$ may be possible. Such an algorithm, if it exists, may perform well under any scene, as its path lengths can not be large. The new algorithm *ALg1* fulfills these expectations as will be seen soon.

4.2. A Summary of Algorithm Alg1

Alg1 can be briefly described as follows: *MA*, starting at S moves along line segment ST until an obstacle is met. Then, it starts following the obstacle boundary in the prespecified local direction (i.e. left). The hit point H_j (j denotes the j_{th} occurrence of *MA* meeting or leaving an obstacle) and the length along the path, $d_p(S, H_j)$ are stored together. *MA* leaves the obstacle boundary at a point L on line segment ST, if and only if, the following two conditions are satisfied.

(1) *MA* can move along line segment LT.

(2) L is the closest point to T on line segment ST, ever visited by *MA*.

The leave point L and the path length from S up to L are stored together. *MA*, after leaving an obstacle boundary at L, moves along line segment LT until an obstacle is met again or target T is reached. If an obstacle is met, the above procedure is reapplied. Up to now, except for storing of the path lengths, the procedure has been the same as in *Bug2*. Hence, *Alg1* behaves like *Bug2* for this part. But, when local cycles are encountered (i.e., *MA* revisiting a point on its path) *Alg1* behaves differently. The new idea behind *Alg1's* action in this situation is as follows:

> When *MA* meets an already visited point on an obstacle boundary (clearly, this must be a previously defined hit or a leave point), then the section of the boundary ahead also must have been visited before. At this juncture, *MA's* goal is to reach an unvisited obstacle boundary section, for seeking the target T. Since, the boundary sections ahead and behind its current position have been already visited before, *MA* finds the shortest path through these visited sections (thus minimizing repeat trips), to reach an unvisited section. One such unvisited section starts from the last defined hit point. Hence, *MA*, returns to this last defined hit point, through the shortest possible path along the already visited path sections. The path length information stored with each hit or a leave point is used to find this shortest path section to return to the last defined hit point. After returning to the last defined hit point, it enters into the unvisited boundary section starting from that hit point. This results in an overall reduction in the path length leading to a smaller worst case path length. Actually, the choice of returning to the last defined hit point is optimal in that it results in the minimum possible worst case path length. This is elaborated in Sankar [13]. The next part describes this operation of *Alg1*.

After meeting an obstacle at hit point H_j and moving along that obstacle boundary, *MA* can meet a previously defined hit or leave point Q_k ($k < j$). If that happens, *MA* retraces its path back to H_j (the last defined hit point). After returning to H_j, it continues to move along the section of the obstacle boundary on the other side of H_j, which has not been visited by *MA*, so far. There are at least two path sections through which *MA* can return to H_j from Q_k. The path length information stored with each hit and leave point is used to select the shorter of these two available paths.

If either S or T is trapped inside an obstacle, then T is unreachable. The algorithm determines this by checking whether a certain condition is satisfied.

4.3. The Procedure for Alg1

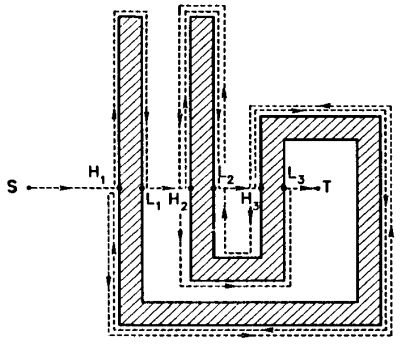

Fig 1. A Scene to illustrate algorithm *Alg1's* procedure

The algorithm can be followed using the example shown in Fig. 1. The goal is to generate a continuous path from S to T. The algorithm is to be executed at any point on the path. The hit points and the leave points are arranged in a sequence in the order of their occurrences. The subscript j denotes the j_{th} occurrence of a hit, or a leave point. The starting point S is considered as leave point L_0 and the leave point L_j occurs after the hit point H_j. The following two pieces of information are stored along with each hit point H_j:

(1) The distance $d_p(S, H_j)$, which is the distance along the path from S to the concerned hit point H_j.

(2) The local direction at H_j. This is initially set as "left" for all hit points. This could be changed to "right" under certain conditions, as described in the algorithm. If, *MA* ever meets the hit point H_j again, while moving along the line segment ST, it then follows the obstacle boundary at H_j along the local direction stored with H_j.

Along with each leave point L_j, the distance $d_p(S, L_j)$ is stored. From the ordering of the hit and the leave points, it is possible to identify the leave point L_j, which occurs immediately after a hit point H_j.

To determine the unreachability of target T, a certain leave point is stored in a variable U. The path length from S up to the current MA location is integrated and stored in the path length counter PC. The shortest distance between MA and T, when MA is on line segment ST, is stored in a variable R.

The algorithm consists of the following steps:

(0) Set $j = 1$, $L_{j-1} = S$, $PC = 0$ and $R = D$.

(1) From L_{j-1}, move along the line segment $L_{j-1}T$ until one of the following occurs:

(1a) Target T is reached. The procedure stops.

(1b) An obstacle is met. Define a hit point H_j. Store the distance $d_p(S, H_j)$ from path counter PC, and the local direction "left" along with H_j. Set $R = d(H_j, T)$. Turn left and follow the obstacle boundary. Go to 2.

(2) Move along the obstacle boundary until one of the following occurs.

(2a) Target T is reached. The procedure stops.

(2b) Line segment ST is met at P, such that $d(P, T) < R$.

(2b1) If MA is free to move along the line segment PT, define a leave point L_j and store the distance $d_p(S, L_j)$ along with it. Set $j = j + 1$. Go to 1.

(2b2) If MA cannot move along PT, set $R = d(P, T)$. Go to 2.

(2c) A hit or a leave point Q_k $(k < j)$, is met. Let d_c be the content of the path length counter PC at Q_k. The following steps are executed.

The local direction at H_j is reset to "right".

If Q_k is a hit point H_k, then the coordinates of leave point L_k, which occurs immediately after H_k, are stored in U.

The quantities d_1 and d_2 are calculated as follows.

$$d_1 = d_p(S, H_j) - d_p(S, Q_k) \quad (3)$$

$$d_2 = d_c - d_p(S, H_j) \quad (4)$$

Here, $d_p(S, H_j)$ and $d_p(S, Q_k)$ are the path length information stored with the points H_j and Q_k, respectively.

Reset the content of PC to $d_p(S, H_j)$. Stop incrementing PC until MA returns to hit point H_j from its current location at Q_k.

The path sections d_1 and d_2 are compared next.

(2c1) If $(d_1 \geq d_2)$,

Reverse the direction of MA's motion and proceed along obstacle boundary. After returning to H_j, MA continues to move along the obstacle boundary in the same direction. Start incrementing PC. Go to 2c3.

(2c2) If $(d_1 < d_2)$,

MA moves along the same path section already generated between the points Q_k (i.e., H_k or L_k) and H_j, when MA passed Q_k, the first time. That is, MA will pass through all the previously defined hit and the leave points between Q_k and H_j.

MA moves along obstacle boundary from Q_k (without reversing its direction). If a previously defined leave point L_i $(k \leq i < j)$ is met, MA leaves obstacle boundary and moves along line segment L_iT. This takes it to the previously defined hit point H_{i+1}. At H_{i+1}, MA checks the local direction stored with the hit point and moves along obstacle boundary accordingly. This procedure of retracing the previously defined hit and the leave points takes MA back to H_j. At H_j, MA moves along obstacle boundary in the local direction stored with H_j (i.e., right). The path counter PC is started again. Go to 2c3.

(2c3) Continue along obstacle boundary until one of the following occurs:

(a) Target T is reached. The procedure stops.

(b) Line segment ST is met at P such that $d(P, T) < R$.

(b1) If MA can move along PT, define a leave point L_j and store the content of PC (i.e. $d_p(S, L_j)$) along with it. Set $j = j + 1$. Go to 1.

(b2) If MA cannot move along PT, set $R = d(P, T)$. Go to 2c3.

(c) MA reaches a point stored in U. Target T is not reachable as either S or T is trapped inside an obstacle. The procedure stops.

(2d) MA returns to H_j without defining any leave point. Again, target T is unreachable. The procedure stops.

4.4. Comments on Steps of Algorithm Alg1

The steps 1, $2a$, $2b1$ and $2d$, excluding the path length storing, form the existing algorithm *Bug2*.

The Step $2c$, which forms the bulk of the new algorithm, is a new contribution of algorithm *Alg1*. This step becomes active only when *MA* meets a previously defined hit point or a leave point (i.e., when local cycles occur). The basic operation of this step is to take *MA* back to its last defined hit point, when a previously defined hit or a leave point is met. It also optimizes this operation by taking *MA* back along the shorter of the two available path sections.

The operation of step $2c$ can be explained through the example in Fig. 1. *MA*, during its trip from S to T, meets the previously defined hit point H_1 after defining the hit point H_3. The first part of $2c$ finds the quantities d_1 and d_2, which are the lengths of the two path sections $H_1L_1H_2L_2H_3$ and H_1H_3, respectively. These path sections, which can take *MA* from H_1 back to H_3, are computed using the path length stored with each hit or a leave point. The algorithm chooses the shorter of these two path sections, which is d_2 in this case. Then, Step $2c1$ generates the path along the section H_1H_3. Step $2c3$ generates *MA's* path after it returns to H_3.

The step $2b$ is repeated, inside step $2c$, as step $2c3$. This is done to prevent *MA* from getting into an infinite loop. The example in Fig. 1, clarifies this point also. *MA*, after returning to H_3, proceeds along the obstacle boundary beyond H_3 to meet the leave point L_2. Applying the step $2c$ again (because *MA* has met a previously defined leave point) and returning to the hit point H_3 will put *MA* into an infinite loop. Hence, the step $2c3$, which controls *MA's* motion after it returns to H_3, ignores such meetings. Only after *MA* defines a new leave point L_3, can the step $2c$ be applied again.

The terminating condition $2d$ is the same as in *Bug2*. An additional terminating condition $2c3.c$ is also needed. If either one of these conditions is satisfied, then target T can not be reached. The rationale behind this new condition will be discussed later.

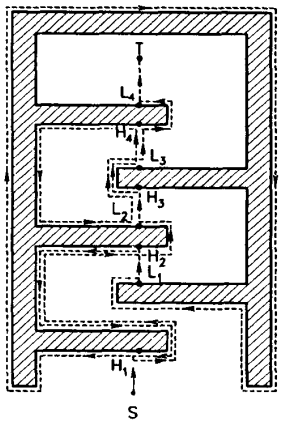

Fig. 2. An example of *Alg's* path generation

5. EXAMPLES

The path P_{Alg1} generated by *Alg1*, for a scene consisting of a single obstacle, is shown in Fig. 2.

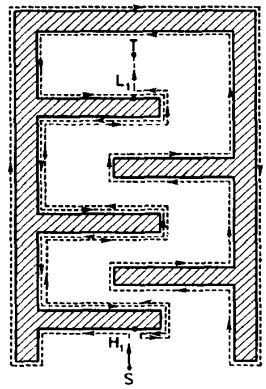

Fig. 3. The path generated by *Bug1* for the scene in Fig. 2

The path generated under *Bug1* for the same scene is shown in Fig. 3. The path P_{Alg1} seems to be shorter than P_{Bug1}. Fig. 4 shows the path generated by algorithm *Bug2* for the same scene.

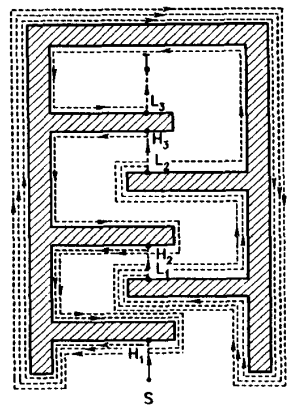

Fig. 4. The path generated by *Bug2* for the scene in Fig. 2

The path P_{Bug2} is much longer than P_{Alg1}, because *MA* repeats its trip many times along long path sections.

6. CHARACTERISTICS OF ALG1

Lemma 1: *Under Alg1, MA on its way from S to T can meet only a finite number of obstacles.*

Proof: Same as given by Lumelsky and Stepanov [9] for *Bug2*. ■

To compare *Alg1* and *Bug2*, it is necessary to relate their capacity to create local cycles.

Lemma 2: *For any given scene, a local cycle is created by Alg1, if and only if Bug2 creates one.*

Proof: Assume that *Bug2* does not create a local cycle for a given scene. If *Alg1* is applied to the same scene, the step 2*c* will not be invoked at all as *MA*, by hypothesis, can not meet a point already visited by *MA*. Since the rest of *Alg1* without step 2*c* is same as *Bug2*, the paths generated by both the algorithms are identical. That is, *Alg1* also does not create a local cycle for the given scene.

In a similar way, it can be shown that if *Alg1* does not create a local cycle for a given scene, *Bug2* also won't create one. ■

It is necessary to establish that *Alg1* indeed takes *MA* to *T* and does not produce any infinite loops in *MA's* path. This is equivalent to showing that, under *Alg1*, for any scene, *MA* travels only a finite length to reach *T*, or to conclude that *T* is not reachable. That is, *Alg1* must have a finite upper bound on its path length. The upper bound on the path length generated by *Alg1*, in terms of obstacle perimeters and the length *D* of line segment *ST*, is also essential to compare its performance with the existing algorithms.

6.1. Algorithm Alg2

To determine the upper bound M_{Alg1}, a variation of algorithm *Alg1*, called *Alg2* is first formulated. Algorithm *Alg2* differs from *Alg1* only in step 2*c*. Under *Alg1*, when *MA* meets a previously defined hit or leave point *Q*, it retraces its path back to the last defined hit point *H* (Step 2*c*), by taking the shorter of two possible path sections from *Q* to *H*. One of these path sections is generated just by reversing *MA's* direction at *Q* and follow the obstacle boundary until *H* is reached. Under *Alg2*, after meeting *Q*, *MA* always chooses this path section. Hence, the path from *Q* and to *H* under *Alg1* will be always shorter than or equal to that produced under *Alg2*. The paths generated by *Alg1* and *Alg2* are identical everywhere else, except in those sections where *MA* goes back from *Q* to *H*. Hence, the path length L_{Alg1} is always less than or equal to L_{Alg2} for any scene. The worst case path length under *Alg2* is determined first. The following Lemma is stated without proof for space considerations. The proof is long because of the various cases to be considered. The basic approach is to show that, under algorithm *Alg2*, *MA* does not visit any point on its path more than twice.

Lemma 3: *Under Alg2, the path length from S to T cannot exceed the limit,*

$$M_{Alg2} = 2\ \Sigma p_i + D, \tag{5}$$

where p_i is the perimeter of the i_{th} obstacle met by MA and D is the length of line segment ST.

The following corollary, characterizes the new unreachability condition. The proof of this corollary is contained in the proof of Lemma 3.

Corollary 1: *Let MA meet a previously defined hit point H_k after defining a hit point H_j $(j > k)$. Then, after returning to H_j, if MA ever meets the leave point L_k (which was defined immediately after H_k), then the target T is unreachable.*

The following theorem establishes that the upper bound M_{Alg2} is indeed the tight bound for *Alg1* too. That is, under *Alg1*, a path length arbitrarily close to this upper bound can be achieved.

Theorem 1: *Under Alg1, for any scene, the path length from S to T cannot exceed the limit,*

$$M_{Alg1} = 2\ \Sigma p_i + D. \tag{6}$$

Moreover, for any $\epsilon > 0$ (however small), there always exists a scene such that the path length L_{Alg1} is given by,

$$L_{Alg1} = 2\ \Sigma p_i + D - \epsilon. \tag{7}$$

Proof: Since $L_{Alg1} \leq L_{Alg2}$, it follows from Lemma 3 that,

$$M_{Alg1} \leq M_{Alg2} = 2\ \Sigma p_i + D \tag{8}$$

To prove the second part, let us construct a scene for a given $\epsilon > 0$, such that the required path length is obtained.

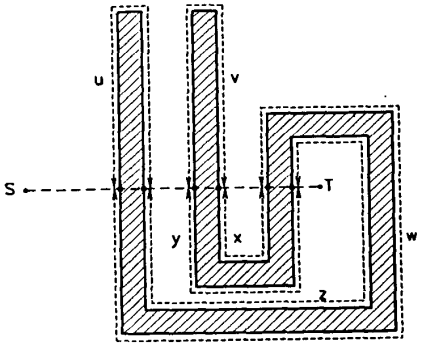

Fig. 5. An obstacle construction for proving theorem 1

Consider the scene shown in Fig. 1. Fig. 5 shows the same scene with the lengths of different sections of obstacle boundary marked. The perimeter *p* of the obstacle is,

$$p = u + v + w + x + y + z, \tag{9}$$

where the lengths *u* to *z* can be arbitrarily chosen. Let the uniform thickness of the obstacle be *t*.

Assume that *v* is greater than *w*. This results in the path shown in Fig. 1. The length *L* of the path is,

$$L = u + 2v + 2w + x + y + D - 3t. \tag{10}$$

The term $3t$ is due to the fact that *MA* did not travel on *ST* at those sections where the obstacle intersected *ST*. Substituting for $v + w$ from Eqn. 9,

$$L = 2p - (u + x + y + 3t + z) + D \tag{11}$$

Since u, x, y, z and t can be arbitrarily chosen (i.e. any choice of their values will yield the same path as long as $v > w$), making,

$$\epsilon = u + x + y + z + 3t, \tag{12}$$

gives the required relation,

$$L = 2p + D - \epsilon. \tag{13}$$

Thus, the theorem is proved. ■

6.2. The Optimization Procedure

The optimization procedure in *Alg1* chooses the shorter of the two path sections from a previously defined hit or leave point Q to the last defined hit point H. When the local cycle created touches multiple obstacles, there are more than two path sections available to go from Q to H. It can be shown that, for an n obstacle case there are up to $n+1$ such path sections available to go from Q back to H. An efficient data structure can store these various path sections and use the shortest among them. If provided with such a feature, *Alg1* can produce a shorter path, for a given scene, than in its present form.

6.3. Test for Target Reachability

The algorithm decides the reachability of T by checking whether either of the two conditions 2*c*3.*c* and 2*d* is met. If any one of these conditions is satisfied, then either S, or T is trapped inside an obstacle.

The condition 2*d* is the same as that present in *Bug2*. Under this condition, if *MA* ever returns to a hit point without defining a new leave point, then T is unreachable.

The second condition 2*c*3.*c* is unique to *Alg1*. Under this condition, if *MA* returns to a certain leave point L_k, then T is unreachable. The suffix k here corresponds to the previously defined hit point H_k, met by *MA* after defining a hit point H_j $(j > k)$. This condition follows from Corollary 1, under Lemma 3.

7. COMPARISON WITH EXISTING ALGORITHMS

The paths P_{Alg1}, P_{Bug1} and P_{Bug2} cover different sections of obstacle boundaries. Hence, a comparison of their actual path lengths for a given scene is generally not possible. Examples shown in Figs. 2, 3 and 4 illustrate this point. The comparisons can be made on the basis of worst case path lengths.

Comparison with algorithm *Bug2*: When a scene does not create local cycles, *Alg1* and *Bug2* produce the same path (Lemma 2). When a complex scene creates local cycles, under both *Alg1* and *Bug2* (Lemma 2), the path P_{Alg1} is generally shorter than the path P_{Bug2}. This follows from their worst case path lengths. *Alg1's* worst case path length $2\,\Sigma p_i + D$, is independent of the complexity of the scene. *Bug2's* worst case path length $\Sigma n_i p_i/2 + D$, is quite dependent on the scene. A complex obstacle can make n_i quite large.

The effectiveness of *Alg1* is due to its complete use of all the information available from *MA's* path. Using the path length information stored with each hit or a leave point, *Alg1* minimizes the repeat trips along already traversed path sections. In fact, *Alg1's* worst case path length cannot be reduced any further as it is the minimum possible for any algorithm in which *MA* leaves an obstacle boundary before fully traveling along that. This is proved in Sankar [13].

Comparison with algorithm *Bug1*: When a scene does not create local cycles under *Alg1*, the path P_{Alg1} is generally shorter than the path P_{Bug1}. When there are local cycles under *Alg1*, its worst case path length, $2\,\Sigma p_i + D$ is only slightly greater than *Bug1's* worst case path length, $1.5\,\Sigma p_i + D$. Also, it is observed that the path length L_{Alg1} is generally shorter than L_{Bug1}, for any given scene. It may be that the average path length under *Alg1* is shorter than that under *Bug1*.

8. CONCLUSIONS

The new algorithm *Alg1* performs uniformly well, whether a scene creates local cycles or not. The paths produced by it are generally shorter than those produced by other existing algorithms. This is mainly due to *Alg1's* use of the path information to minimize repeated trips along any path section. The worst case path length (the upper bound on the path length) of *Alg1* is the minimum possible for any algorithm of its type (i.e. *MA* leaves an obstacle boundary before moving along the entire boundary).

9. REFERENCES

[1] *Algorithmic Motion Planning,* C. K. Yap, Advances in Robotics, Vol. 1: Algorithmic and Geometric Aspects (J. T. Schwartz and C. K. Yap Eds.), 1987.

[2] *On the "Piano Movers" Problem: 1. The Case of a Two-dimensional Rigid Polygonal Body Moving Amidst Polygonal Barriers,* J.T. Schwartz and M. Sharir, Comm. Pure Appl. Math., 36, 1983.

[3] *Spatial Planning: A Configuration Space Approach,* Tomas Lozano-Perez, IEEE Transactions on Computers, Vol. C-32, No. 2, 1983.

[4] *Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,* O. Khatib, The International Journal of Robotics Research, Vol. 5, No. 1, 1986.

[5] *Obstacle Avoidance for Kinematically Redundant Manipulators in Dynamically Varying Environments,* A. A. Maciejewski and C. A. Klein, The International Journal of Robotics Research, Vol. 4, No. 3, 1985.

[6] *Contribution to Control of Redundant Robotic Manipulators in an Environment with Obstacles,* M. Kircanski and M. Vukobratovic, The International Journal of Robotics Research, Vol. 5, No. 4, 1986.

[7] *Some Heuristics for the Navigation of a Robot,* R. Chattergy, The International Journal of Robotics Research, Vol.4, No. 1, 1985.

[8] *Turtle Geometry,* H. Abelson and A. diSessa, MIT Press, Cambridge, MA, 1980, pp. 176-199.

[9] *Path Planning Strategies For A Point Mobile Automaton Moving Amidst Unknown Obstacles Of Arbitrary Shape,* V. J. Lumelsky, A. A. Stepanov, Algorithmica, Springer-Verlag, 1987.

[10] *Dynamic Path Planning For A Mobile Automaton With Limited Information On The Environment,* V. J. Lumelsky, A. A. Stepanov, IEEE Transactions On Automatic Control, Vol. AC. 31, No. 11, Nov. 1986.

[11] *Dynamic Path Planning For A Planar Articulated Robot Arm Moving Amidst Unknown Obstacles,* V. J. Lumelsky, Automatica, Journal Of International Federation Of Automatic Control, June 1987.

[12] *Effect of Kinematics On Dynamic Path Planning For Planar Arms Moving Amidst Unknown Obstacles,* V. J. Lumelsky, IEEE Journal of Robotics And Automation, May, 1987.

[13] *The Universal Lower Bound On Worst Case Path Lengths Of Path Planning Algorithms For Moving A Point Object Amidst Unknown Obstacles In A Plane,* A. Sankaranarayanan, Chapter 2 of Ph. D. Thesis (In Preparation), Electrical Engineering Department, University Of Waterloo, Canada.